

Usando DAX en Power BI Desktop

(Adaptación y traducción libre del tutorial DAX de Microsoft Power BI)

DAX (Data Analysis eXpressions) es un conjunto de instrucciones para realizar cálculos que permiten mejorar el modelo de datos Power BI. DAX permite construir fórmulas y otros tipos de cálculos como tablas, columnas y medidas calculadas

Modelos Power BI Desktop

Para construir un modelo de datos Power BI debemos tener en cuenta cuáles son las preguntas de los usuarios a las que las visualizaciones del modelo deben responder. El proceso del diseño del modelo de datos pasa por las siguientes etapas:

- Conexión a los datos.
- Transformación y preparación de los datos.
- Definición de la lógica de negocios que soportará los cálculos DAX.
- Publicación del modelo Power BI.

El modelo recomendado para Power BI está basado en el esquema **estrella** formado por la tabla de hechos y sus medidas, y las tablas de dimensiones.

Tema A: Visualizaciones Power BI y consultas analíticas

Consultas analíticas

Una consulta analítica es una consulta DAX que produce un resultado a partir de un modelo de datos. La consulta analítica puede pasar por 3 fases que se ejecutan en el siguiente orden:

1. **Filtrado:** obtiene los datos relevantes para el usuario del informe. Los filtros se pueden aplicar a nivel del informe, a nivel de la página o a nivel de la visualización.
2. **Agrupación:** separa los resultados de la consulta en grupos.
3. **Resumen:** genera un valor único para cada grupo obtenido en el paso anterior. Utiliza las funciones de agregación o medidas creadas con DAX.

Configurar una visualización

Para diseñar una visualización debemos ejecutar los siguientes pasos:

1. Seleccionar el tipo de visualización.
2. Desde el panel **Datos** asignar los campos del conjunto de datos a los componentes de la visualización.
3. Configurar los campos de la visualización.
4. Aplicar opciones de formato.

Ejercicio: añadir una visualización a un informe Power BI

Abra el archivo **Adventure Works DW 2020 ModA** de Power BI. Añada a una página del informe un gráfico de columnas apiladas que muestre el monto de las ventas mensuales para el año fiscal 2020.

1. **Vista de informe**, página **Page 1**, añada **Gráfico de columnas apiladas**.
2. **Fase de filtrado:** panel **Filtros**, **Filtros de esta página**, tabla **Date** columna **Fiscal Year**.
3. Selecciona **FY2020**.
4. **Fase de agrupación:** panel **Visualizaciones**, campo **Eje X**, tabla **Date** columna **Month**.
5. **Fase de resumen:** panel **Visualizaciones**, campo **Eje Y**, tabla **Sales** columna **Sales Amount**.

Tema B: Tablas calculadas y columnas calculadas

Tablas calculadas

Es una tabla cuyo contenido se establece mediante una fórmula DAX. La tabla calculada se añade al modelo por lo que afecta el tamaño del almacenamiento de datos y el tiempo de actualización de los datos del modelo.

Las tablas calculadas pueden ser usadas en distintos escenarios como

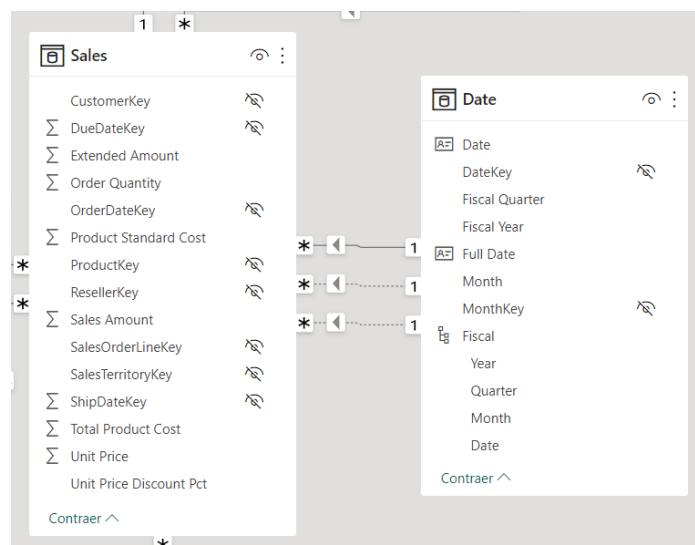
- Tablas de fechas
- Dimensiones con juego de roles
- Análisis "Qué pasa si"

Tablas de fechas

Las tablas de fechas son necesarias para realizar análisis de datos por periodos de tiempo. DAX posee una categoría de funciones conocida como funciones de inteligencia de tiempo que requieren para su uso de la existencia de tablas de fechas. Puede crear una tabla de fechas usando las funciones DAX CALENDAR o CALENDARAUTO.

Dimensiones con juego de roles

Cuando dos tablas del modelo tienen varias relaciones, podría deberse a que su modelo tiene una dimensión con juego de roles. Por ejemplo, si tiene una tabla denominada **Sales** que incluye tres columnas de fecha, **OrderDateKey**, **ShipDateKey** y **DueDateKey**, las tres columnas están relacionadas con la columna **Date** en la tabla **Date**. En este caso, la tabla **Date** se describe como una dimensión con juego de roles porque podría desempeñar el papel de la fecha del pedido, o la fecha de envío, o la fecha de vencimiento.



Los modelos de Power BI solo permiten una relación activa entre tablas, que en el diagrama del modelo se indica como una línea continua. La relación activa se utiliza de forma predeterminada para propagar filtros, que en este caso sería de la tabla **Date** a la columna **OrderDateKey** en la tabla **Sales**. Las relaciones restantes entre las dos tablas están inactivas. En un diagrama del modelo, las relaciones inactivas se representan como líneas discontinuas. Las relaciones inactivas solo se utilizan cuando se solicitan expresamente en una fórmula calculada mediante la función DAX USERELATIONSHIP.

Un mejor diseño del modelo podría tener tres tablas de fechas, cada una con una relación activa con la tabla de **Sales**. Por lo tanto, los usuarios de informes pueden filtrar por fecha de pedido o fecha de envío o fecha de vencimiento, o las tres fechas al mismo tiempo. Una tabla calculada puede duplicar los datos de la tabla **Date** para crear la tabla **Ship Date**.

Análisis "Qué pasa si"

Power BI Desktop admite una característica denominada parámetros hipotéticos. Cuando crea un parámetro hipotético, se agrega automáticamente una tabla calculada a su modelo.

Los parámetros hipotéticos permiten a los usuarios del informe seleccionar o filtrar por valores almacenados en la tabla calculada. Las fórmulas de medida pueden utilizar los valores seleccionados de forma significativa. Por ejemplo, un parámetro hipotético podría permitir al usuario del informe seleccionar una tasa de cambio de moneda hipotética, y una medida podría dividir los valores de ingresos (en una moneda local) por la tasa seleccionada.

En particular, las tablas hipotéticas calculadas no están relacionadas con otras tablas modelo porque no se utilizan para propagar filtros. Por esta razón, a veces se les llama *tablas desconectadas*.

Ejercicio: crear una tabla calculada duplicando otra tabla

Abra el archivo **Adventure Works DW 2020 ModB** y vaya a la vista **Modelo**. La tabla **Sales** tiene 3 relaciones con la tabla **Date** ya que para cada venta se tiene 3 fechas: la fecha del pedido (**OrderDateKey**), la fecha de envío (**ShipDateKey**) y la fecha de vencimiento (**DueDateKey**). Recuerde que la línea continua indica cuál es la relación activa. En el modelo la relación activa es por la fecha del pedido (**OrderDateKey**) por lo que los filtros de las ventas solo se podrán realizar por la fecha del pedido y no por la fecha de envío o por la fecha de vencimiento.

Para que sea posible filtrar por la fecha de envío se requiere que su relación esté activa, por lo que ayudaría crear un duplicado de la tabla **Date** bajo el nombre **Ship Date** para que esta última defina la relación con la columna **ShipDateKey** de **Sales**.

Duplicar la tabla Date

1. **Vista de modelo**, elimina las relaciones inactivas entre las tablas **Sales** y **Date**.
2. **Cinta de opciones**, grupo Cálculos, Nueva tabla.
3. **Barra de fórmula**, digita

```
Ship Date = 'Date'
```

4. Pulsa [Enter].
5. Relaciona la tabla **Ship Date** con la tabla **Sales** uniendo **ShipDateKey** de Sales con **DateKey** de **Ship Date**.

La creación de una tabla calculada solo duplica la data; no duplica la configuración de la tabla por lo que podría ser necesario configurarla de acuerdo con las necesidades de información de los usuarios del informe.

Configurar la tabla Ship Date

1. Renombra las siguientes columnas de **Ship Date**:
 - Date a Ship Date
 - Fiscal Year a Ship Fiscal Year
 - Fiscal Quarter a Ship Fiscal Quarter
 - Month a Ship Month
 - Full Date a Ship Full Date
2. Ordena la columna **Ship Full Date** por la columna **Ship Date**: para ello, **Vista de datos**, tabla **Ship Date**, columna **Ship Full Date**, **Cinta de opciones**, grupo **Ordenar**, **Ordenar por columna**, **Ship Date**.
3. Ordena la columna **Ship Month** por la columna **MonthKey**.
4. Oculta la columna **MonthKey**.
5. Crea una jerarquía de nombre **Fiscal** con los niveles:
 - Ship Fiscal Year
 - Ship Fiscal Quarter
 - Ship Month
 - Ship Full Date
6. Marca la tabla **Ship Date** como una tabla de fechas usando la columna **Ship Date**: para ello, Panel **Datos**, clic secundario tabla **Ship Date**, **Marcar como tabla de fechas**, **Marcar como tabla de fechas**, columna **Ship Date**, clic en **Aceptar**.

Ejercicio: crear una tabla de fechas usando la función DAX CALENDARAUTO

Se desea crear una tabla **Due Date** que se relaciona con la tabla **Sales** de modo que se pueda filtrar datos por la columna **DueDateKey** de la tabla **Sales**.

Crear la tabla Due Date

1. **Vista de datos**, menú **Herramientas de tablas**, **Cinta de opciones**, grupo **Cálculos**, **Nueva tabla**.
2. **Barra de fórmula**, digite la fórmula:

Due Date = `CALENDARAUTO(6)`

3. Pulsa [Enter].

El valor 6 del argumento especifica que el año termina en el mes de junio (el 30 de junio). Si no se pasa el argumento se asume que el año termina el 31 de diciembre.

La función CALENDARAUTO recorre todas las columnas de tipo fecha y fecha-hora del modelo de datos para identificar las fechas más antiguas y más recientes almacenadas en el modelo. Con dichos valores construye un calendario que contiene años completos.

Crear columnas calculadas

Completar la tabla Due Date creando columnas calculadas

1. Renombra la columna **Date** a **Due Date**.
2. Añade la columna calculada **Due Fiscal Year**.

```
Due Fiscal Year =  
    "FY"  
    & YEAR('Due Date'[Due Date])  
    + IF(  
        MONTH('Due Date'[Due Date]) > 6,  
        1  
    )
```

3. Añade la columna **Due Fiscal Quarter**.

```
Due Fiscal Quarter =  
'Due Date'[Due Fiscal Year] & " Q"  
    & IF(  
        MONTH('Due Date'[Due Date]) <= 3,  
        3,  
        IF(  
            MONTH('Due Date'[Due Date]) <= 6,  
            4,  
            IF(  
                MONTH('Due Date'[Due Date]) <= 9,  
                1,  
                2  
            )  
        )  
    )
```

4. Agrega la columna **Due Month**.

```
Due Month =  
FORMAT('Due Date'[Due Date], "yyyy mmm")
```

5. Agrega la columna **Due Full Date**.

```
Due Full Date =  
FORMAT('Due Date'[Due Date], "yyyy mmm, dd")
```

6. Añade la columna **MonthKey**.

```
MonthKey =  
(YEAR('Due Date'[Due Date]) * 100) + MONTH('Due Date'[Due Date])
```

Completar la configuración de la tabla Due Date

1. Ordena la columna **Due Full Date** en base a la columna **Due Date**.
2. Ordena la columna **Due Month** en base a la columna **MonthKey**.
3. Oculta la columna **MonthKey**.
4. Crea una jerarquía de nombre **Fiscal** con los niveles:
 - Due Fiscal Year
 - Due Fiscal Quarter
 - Due Month
 - Due Full Date
5. Marca la tabla **Due Date** como una tabla de fechas usando la columna **Due Date**.

Cómo se evalúa la fórmula de una columna calculada

La fórmula de una columna calculada se evalúa para cada fila de la tabla; es decir que el contexto de la fórmula es la fila actual. Sin embargo, este contexto de fila no se aplica a las tablas relacionadas con la tabla actual. Si la fórmula necesita hacer referencias a las columnas de las tablas relacionadas se debe emplear las funciones RELATED o RELATEDTABLE. Si las tablas no están relacionadas se puede usar la función LOOKUPVALUE.

Ejercicio: crear un campo calculado que hace referencia a columnas de una tabla relacionada

En la tabla **Sales** crear la columna calculada **Discount Amount** que calcula el monto del descuento aplicado a un producto vendido:

```
Discount Amount =  
(  
Sales[Order Quantity] * RELATED('Product'[List Price])  
) - Sales[Sales Amount]
```

Tema C: Medidas

Crear medidas

Abra el archivo **Adventure Works DW 2020 ModC**. En su modelo de datos vamos a crear medidas que realicen cálculos con la data del modelo, y que luego serán utilizadas en el informe Power BI.

Añadir medidas implícitas a una visualización

1. **Vista de informe**, página **Page 1**.
2. Agrega a la visualización la columna **Sales Amount** de la tabla **Sales**. ¿Cuál es la función de agregación aplicada a los valores de la columna **Sales Amount**?
3. Agrega a la misma visualización la columna **Unit Price**. ¿Cuál es la función de agregación aplicada?

Crear medidas simples

Una medida simple es aquella que resume o consolida los valores de una sola columna.

1. Añade a la tabla **Sales** las siguientes medidas (formatea apropiadamente los valores de las medidas):

Revenue = `SUM(Sales[Sales Amount])`

Cost = `SUM(Sales[Total Product Cost])`

Profit = `SUM(Sales[Profit Amount])`

Quantity = `SUM(Sales[Order Quantity])`

Minimum Price = `MIN(Sales[Unit Price])`

Maximum Price = `MAX(Sales[Unit Price])`

Average Price = `AVERAGE(Sales[Unit Price])`

2. Agrega a la tabla **Sales** dos medidas: una para contar la cantidad de pedidos, y la segunda para contar la cantidad de ítems de pedidos. Formatea sus valores adecuadamente.

Order Count = `DISTINCTCOUNT('Sales Order'[Sales Order])`

Order Line Count = `COUNT(Sales[SalesOrderLineKey])`

3. Agrega a la visualización la medida **Revenue**, y remueve los campos **Sales Amount** y **Unit Price**.

Crear una medida compuesta

Una medida compuesta es aquella que hace referencia a una o más medidas.

Modifique la medida **Profit** de la tabla **Sales** usando la siguiente fórmula.

$$\text{Profit} = [\text{Revenue}] - [\text{Cost}]$$

La medida **Profit** entrega el mismo resultado que la columna calculada **Profit Amount**, pero lo hace sin consumir mucho espacio de almacenamiento y optimizando el tiempo requerido para refrescar la data. Podemos eliminar la columna calculada **Profit Amount**.

Crear una medida rápida

La medida rápida es una característica de Power BI que permite crear rápidamente cálculos comunes y de uso frecuente.

Se requiere una medida que permita establecer el margen de utilidad de las ventas. La denominación de la medida es **Profit Margin**.

1. Panel **Datos**, tabla **Sales**.
2. Menú **Herramientas de tablas**, **Cinta de opciones**, grupo **Cálculos**, **Medida rápida**.
3. Panel **Medida rápida**, pestaña **Cálculos**.
4. Lista desplegable **Seleccionar un cálculo**, **Operaciones matemáticas**, **División**.
5. **Numerador**, columna **Profit** tabla **Sales**.
6. **Denominador**, columna **Revenue** tabla **Sales**.
7. Clic en **Agregar**.
8. Medida **Profit dividido por Revenue**, renombrar a **Profit Margin**, formatear a **Porcentaje** con 2 decimales.
9. Añade la medida a la visualización.

Tema D: Funciones de iteración

Usar funciones de iteración

Una función de iteración evalúa una expresión dada para cada fila de un conjunto. La expresión debe entregar un escalar.

Las funciones de agregación de una sola columna, como SUM son funciones abreviadas. Internamente, Power BI convierte la función SUM en la función de iteración SUMX. Las siguientes fórmulas DAX producen el mismo resultado.

```
Revenue = SUM(Sales[Sales Amount])
```

```
Revenue = SUMX(Sales, Sales[Sales Amount])
```

Cada función de agregación de una sola columna tiene su función de iteración equivalente. A continuación se muestran dos escenarios de agregación en el que las funciones de iteración son útiles: resumen complejo y resumen de granularidad superior.

Resumen complejo

1. Abre el archivo **Adventure Works DW 2020 ModD**.
2. Agrega a la tabla **Sales** la medida:

```
Revenue =  
SUMX(  
Sales,  
Sales[Order Quantity] * Sales[Unit Price] * (1 - Sales[Unit Price Discount Pct])  
)
```

3. **Vista de informe**, página **Page 1**, agrega la medida **Revenue** a la visualización.
4. Agrega a la tabla **Sales** la medida:

```
Discount =  
SUMX(  
Sales,  
Sales[Order Quantity] *  
    (RELATED('Product'[List Price]) - Sales[Unit Price])  
)
```

5. **Vista de informe**, página **Page 1**, agrega la medida **Discount** a la visualización.

Resumen de granularidad superior

Se desea mostrar en la visualización una columna que calcule el promedio de los montos parciales de los items de cada venta.

1. Agrega a la tabla **Sales** la medida:

```
Revenue Avg Order Line=  
AVERAGEX(Sales,  
Sales[Order Quantity] * Sales[Unit Price] * (1 - Sales[Unit Price Discount Pct])  
)
```

2. Añade la medida a la visualización. En la tabla **Sales** cada fila representa un item de una venta, por lo que el promedio calculado es el promedio de todas las filas correspondientes a las ventas de un mes.
3. Defina una nueva medida que eleva la granularidad al nivel de la venta. Cada venta puede contener varios items de venta.

```
Revenue Avg Order =  
AVERAGEX(VALUES('Sales Order'[Sales Order]), [Revenue])
```

4. Añade la medida a la visualización. En la tabla **Sales Order** cada fila representa un item de una venta. La función VALUES hace referencia a una venta, por lo tanto, el promedio se calcula iterando sobre cada venta. Por ello, el resultado es superior al de la medida anterior que calcula el promedio de los items de una venta. La función VALUES retorna una tabla de una sola columna con los valores distintos en la columna **Sales Order**.

Crear rankings

Para crear un ranking podemos utilizar la función DAX RANKX cuya sintaxis es la siguiente:

```
RANKX(tabla, expresión [, valor][, ordenamiento][, empate]]])
```

Vamos a crear un ranking que muestre las unidades vendidas de cada modelo de bicicleta. Para ello trabajaremos en la visualización mostrada en la página 2 del informe. La visualización muestra en una tabla las ventas en unidades de cada modelo de bicicleta

1. Añade al modelo la medida:

```
Product Quantity Rank =  
RANKX(ALL('Product'[Product]), [Quantity])
```

La función RANKX debe utilizar la función ALL para que tome en cuenta a todos los productos ya que Product es un filtro que se ha definido en la visualización, por lo que si el filtro por Product ha seleccionado solo algunos productos, la función RANKX solo definiría el ranking en base a dichos productos.

2. **Vista de informe**, página **Page 2**, visualización **Bike Sales**, añade la medida **Product Quantity Rank**.
3. El ranking muestra dos productos empatados en el puesto 10, y no toma en cuenta al puesto 11. Esto se debe a que la función RANKX está usando el valor **SKIP** en el argumento **empate**. Además, el **ordenamiento** predeterminado es **DESCENDING**.
4. Redefine la medida para que el argumento **empate** use el valor **DENSE**.

```
Product Quantity Rank =  
RANKX(ALL('Product'[Product]), [Quantity], , , DENSE)
```

Tema E: Filtros

Definir el contexto del filtro

El contexto del filtro determina cómo los filtros son aplicados cuando se evalúa una medida o una expresión asociada a una medida.

Las tablas y columnas calculadas no son evaluadas en base al contexto del filtro. Ellas se evalúan en base al contexto de fila, aunque la fórmula puede pasar del contexto de fila al contexto de filtro si es necesario consolidar datos del modelo.

Para modificar el contexto del filtro cuando una fórmula es evaluada usamos la función DAX CALCULATE. La sintaxis de la función CALCULATE es la siguiente:

```
CALCULATE(expresión, filtro1, filtro2, ...)
```

Aplicar filtros de expresión booleana

Los filtros de expresión booleana deben cumplir las siguientes reglas:

- Solo pueden hacer referencia a una columna.
- No pueden hacer referencia a medidas.
- No pueden usar funciones que recorren o retornan una tabla que incluye funciones de agregación.

1. Abre el archivo **Adventure Works DW 2020 ModE**.
2. Añade una medida que calcule los ingresos por ventas para los productos de color rojo.

```
Revenue Red = CALCULATE([Revenue], 'Product'[Color] = "Red")
```

3. Agrega la medida a la visualización de la página **Page 1** del informe.
4. Define una medida que calcule los ingresos por ventas para los productos de color rojo o verde.

```
Revenue Red or Green = CALCULATE([Revenue], 'Product'[Color] IN {"Red", "Green"})
```

5. Añade una medida que calcule los ingresos por ventas de los productos cuyo precio de lista es superior a USD 1000.

```
Revenue Expensive Products = CALCULATE([Revenue], 'Product'[List Price] > 1000)
```

Aplicar filtros de expresión tabla

Un filtro de expresión tabla aplica un objeto tabla como filtro. Puede hacer referencia a una tabla del modelo, pero lo más común es que sea una función DAX que retorna un objeto tabla. Por lo general se hace uso de la función de iteración FILTER aplicada sobre una tabla y que evalúa una expresión de filtro para cada fila de la tabla. La función FILTER devuelve un subconjunto de las filas de la tabla que recibió.

La siguiente expresión filtra las filas de la tabla **Product** para aquellos productos cuyo precio de lista es superior al costo del producto multiplicado por 2.

```
Revenue High Margin Products =  
CALCULATE( [Revenue],  
    FILTER( 'Product', 'Product'[List Price] > 'Product'[Standard Cost] * 2 )  
)
```

En este ejemplo, la función FILTER filtra todas las filas de la tabla **Product** según el contexto del filtro. Cada una de las filas cuyo precio de lista es superior al doble del costo del producto es mostrada en la tabla filtrada. Luego la medida Revenue es evaluada para todos los productos retornados por la función FILTER.

Tema F: Inteligencia de tiempo

Usar funciones de inteligencia de tiempo

En DAX la inteligencia de tiempo se refiere a la modificación del contexto del filtro para los filtros de fecha. Por ejemplo, en la empresa Adventure Works el año fiscal se inicia el 1 de julio y termina el 30 de junio del año siguiente. Revisemos la siguiente visualización que muestra el ingreso mensual (**Revenue**) y el ingreso anual a la fecha (**Revenue YTD**).

Year	Revenue	Revenue YTD
FY2018	\$23,860,891.17	\$23,860,891.17
2017 Jul	\$1,423,357.32	\$1,423,357.32
2017 Aug	\$2,057,902.45	\$3,481,259.78
2017 Sep	\$2,523,947.55	\$6,005,207.32
2017 Oct	\$561,681.48	\$6,566,888.80
2017 Nov	\$4,764,920.16	\$11,331,808.96
2017 Dec	\$596,746.56	\$11,928,555.52
2018 Jan	\$1,327,674.63	\$13,256,230.15
2018 Feb	\$3,936,463.31	\$17,192,693.45
2018 Mar	\$700,873.18	\$17,893,566.64
2018 Apr	\$1,519,275.24	\$19,412,841.88
2018 May	\$2,960,378.09	\$22,373,219.97
2018 Jun	\$1,487,671.19	\$23,860,891.17
FY2019	\$34,070,108.50	\$34,070,108.50
2018 Jul	\$2,939,691.00	\$2,939,691.00
2018 Aug	\$3,964,801.20	\$6,904,492.20
2018 Sep	\$3,287,605.93	\$10,192,098.13

El contexto del filtro para **Agosto del 2017** contiene cada una de las 31 fechas de agosto, las cuales están almacenadas en la tabla **Date**. Sin embargo, el **ingreso anual a la fecha** para **Agosto del 2017** tiene un contexto de filtro que va desde la primera fecha del año hasta la última fecha en el contexto del filtro; para este ejemplo, va desde el 1 de julio de 2017 hasta el 31 de agosto del 2017.

Aplicar funciones de inteligencia de tiempo

Para trabajar con funciones DAX de inteligencia de tiempo, su modelo de datos tener al menos una tabla de fechas. Esta es una tabla que cumple con los siguientes requisitos:

- Debe tener una columna de tipo de datos Fecha (o fecha / hora), conocida como columna de fecha.
- La columna de fecha debe contener valores únicos.
- La columna de la fecha no debe contener espacios en blanco.
- La columna de la fecha no debe tener fechas perdidas.
- La columna de la fecha debe abarcar años completos. Un año no es necesariamente un año calendario (enero-diciembre).
- La tabla de fechas debe configurarse como tabla de fechas.

Funciones para calcular resúmenes sobre el tiempo

- DATESYTD(colFecha[, fechaFinAño]): retorna una tabla con valores de fecha para "anual a la fecha" en el contexto actual.
- DATESQTD(colFecha): retorna una tabla con valores de fecha para "trimestral a la fecha" en el contexto actual
- DATESMTD(colFecha): retorna una tabla con valores de fecha para "mensual a la fecha" en el contexto actual.
- TOTALYTD(expresión, colFecha[, filtro][, fechaFinAño]): evalúa la expresión para "anual a la fecha" en el contexto actual.
- TOTALQTD(expresión, colFecha[, filtro]): evalúa la expresión para "trimestral a la fecha" en el contexto actual.
- TOTALMTD(expresión, colFecha[, filtro]): evalúa la expresión para "mensual a la fecha" en el contexto actual.
- DATESBETWEEN(colFecha, fechaInicial, fechaFinal): retorna una tabla con valores de fecha dentro del rango especificado.
- DATESINPERIOD(colFecha, fechaInicial, numParteFecha, parteFecha): retorna una tabla con valores de fecha empezando por la fecha inicial hasta el número y tipo de parte de fecha especificados.

Ejemplo

1. Abre el archivo **Adventure Works DW 2020 ModF**.
2. Añade al modelo la siguiente medida que calcula el ingreso anual a la fecha:

Revenue YTD = `TOTALYTD([Revenue], 'Date'[Date], "6-30")`

3. Agrega la medida a la visualización de la página **Page 1**.

Funciones para comparaciones en el tiempo

- DATEADD(colFecha, numParteFecha, parteFecha): retorna una tabla con valores de fecha desplazándose en el tiempo el número y tipo de parte de fecha indicados.
- PARALLELPERIOD(colFecha, numParteFecha, parteFecha): retorna una tabla con valores de fecha que se obtiene desplazando la fecha inicial y la fecha final contenidas en colFecha, y luego llenado el rango entre las dos fechas desplazadas.
- SAMEPERIODLASTYEAR(colFecha): retorna una tabla con valores de fecha desplazándose un año hacia atrás en el tiempo.
- Otras funciones de desplazamiento en el tiempo que retornan una tabla de fecha: NEXTDAY, NEXTMONTH, NEXTQUARTE, NEXTYEAR, PREVIOUSDAY, PREVIOUSMONTH, PREVIOUSQUARTER, PREVIOUSYEAR.

Ejemplo

1. Añade al modelo la siguiente medida que calcula los ingresos del año anterior:

```
Revenue PY =  
VAR RevenuePriorYear = CALCULATE([Revenue], SAMEPERIODLASTYEAR('Date'[Date]))  
RETURN RevenuePriorYear
```

2. Agrega la medida a la visualización de la página **Page 1** del informe.
3. Modifica la medida renombrándola a **Revenue YoY %** y actualizando su cláusula RETURN.

```
Revenue YoY % =  
VAR RevenuePriorYear = CALCULATE([Revenue], SAMEPERIODLASTYEAR('Date'[Date]))  
RETURN  
    DIVIDE(  
        [Revenue] - RevenuePriorYear,  
        RevenuePriorYear  
    )
```

Cálculos adicionales de inteligencia de tiempo

Existen otras funciones de inteligencia de tiempo de DAX que se ocupan de devolver una sola fecha. Aprenderá sobre estas funciones aplicándolas en dos escenarios diferentes.

Las funciones FIRSTDATE y LASTDATE devuelven la primera y la última fecha en el contexto de filtro actual para la columna de fechas especificada.

Calcular nuevas ocurrencias

Otro uso de las funciones de inteligencia del tiempo es contar nuevas ocurrencias. El siguiente ejemplo muestra cómo se puede calcular la cantidad de nuevos clientes para un período de tiempo. Un nuevo cliente se contabiliza en el período de tiempo en el que realizó su primera compra.

1. Añade al modelo la siguiente medida que cuenta el número de clientes distintos "a la fecha". Esto significa contar desde el inicio del tiempo hasta la última fecha en el contexto del filtro actual.

```
Customers LTD =  
VAR CustomersLTD = CALCULATE(  
    DISTINCTCOUNT(Sales[CustomerKey]),  
    DATESBETWEEN('Date'[Date], BLANK(), MAX('Date'[Date])),  
    'Sales Order'[Channel] = "Internet")  
RETURN CustomersLTD
```

2. Añade la medida a la visualización anterior. Observa que la cuenta la calcula hasta el final de cada mes.

La función DATESBETWEEN devuelve una tabla que contiene una columna de fechas que comienza con una fecha de inicio determinada y continúa hasta una fecha de finalización determinada. Cuando la fecha de inicio es BLANK, usará la primera fecha en la columna de la fecha. (A la inversa, cuando la fecha de finalización es BLANK, utilizará la última fecha en la columna de fecha). En este caso, la fecha de finalización está determinada por la función MAX, que devuelve la última fecha en el contexto del filtro. Por lo tanto, si el mes de agosto de 2017 está en el contexto de filtro, la función MAX devolverá el 31 de agosto de 2017 y la función DATESBETWEEN devolverá todas las fechas hasta el 31 de agosto de 2017.

3. Modifica la medida renombrándola a **New Customers** y añade una segunda variable para almacenar la cuenta de clientes distintos antes del periodo de tiempo en el contexto del filtro. La cláusula RETURN resta este valor de la variable **CustomersLTD** para obtener el número de clientes nuevos en el periodo.

```
New Customers =  
VAR CustomersLTD = CALCULATE(  
    DISTINCTCOUNT(Sales[CustomerKey]),  
    DATESBETWEEN('Date'[Date], BLANK(), MAX('Date'[Date])),  
    'Sales Order'[Channel] = "Internet")  
VAR CustomersPrior = CALCULATE(  
    DISTINCTCOUNT(Sales[CustomerKey]),  
    DATESBETWEEN('Date'[Date], BLANK(), MIN('Date'[Date]) - 1),  
    'Sales Order'[Channel] = "Internet")  
RETURN CustomersLTD - CustomersPrior
```

Cálculos instantáneos

En ocasiones, los datos de hechos se almacenan como instantáneas en el tiempo. Los ejemplos comunes incluyen niveles de existencias de inventario o saldos de cuentas. Se carga una instantánea de los valores en la tabla de forma periódica.

Al resumir los valores instantáneos (como los niveles de existencias de inventario), puede resumir los valores en cualquier dimensión excepto la fecha. Agregar recuentos de niveles de existencias en todas las categorías de productos produce un resumen significativo, pero agregar recuentos de niveles de existencias en las fechas no lo hace. Agregar el nivel de existencias de ayer al nivel de existencias de hoy no es una operación útil de realizar (a menos que desee promediar ese resultado).

Al resumir tablas de instantáneas, las fórmulas de medida pueden depender de las funciones de inteligencia de tiempo de DAX para aplicar un solo filtro de fecha.

1. En el siguiente ejemplo, explorará un escenario para la empresa Adventure Works. Cambia a la vista de modelo y selecciona el diagrama de modelo de **Inventory**.

Observa que el diagrama muestra tres tablas: **Product**, **Date** e **Inventory**. La tabla **Inventory** almacena instantáneas de los saldos unitarios para cada fecha y producto. Es importante destacar que la tabla no contiene fechas faltantes ni entradas duplicadas para ningún producto en la misma fecha. Además, el último registro de instantánea se almacena para la fecha del 15 de junio de 2020.

- Ahora, cambia a la vista del informe y selecciona la página 2 del informe. Agrega la columna **UnitsBalance** de la tabla **Inventory** a la visualización matriz. Su agregación predeterminada es la suma de valores.

Product	2019 Jul	2019 Aug	2019 Sep	2019 Oct	2019 Nov	2019 Dec	2020 Jan	2020 Feb	2020 Mar	2020 Apr	2020 May	2020 Jun	Total
Mountain-200 Black, 38	4,884	4,649	4,784	4,873	4,605	4,943	5,208	4,872	5,208	5,040	5,208	2,520	56,794
Mountain-200 Black, 42	5,248	5,175	5,246	5,382	5,200	5,415	5,177	4,843	5,177	5,010	5,177	2,505	59,555
Mountain-200 Black, 46	5,523	5,617	5,287	5,419	5,273	5,663	5,890	5,510	5,890	5,700	5,890	2,850	64,512
Mountain-200 Silver, 38	5,412	5,190	5,114	5,245	5,220	5,406	5,890	5,510	5,890	5,700	5,890	2,850	63,317
Mountain-200 Silver, 42	5,252	5,262	5,043	5,266	5,072	5,208	5,022	4,698	5,022	4,860	5,022	2,430	58,157
Mountain-200 Silver, 46	5,004	5,019	4,960	5,012	4,847	5,231	5,053	4,727	5,053	4,890	5,053	2,445	57,294
Total	31,323	30,912	30,434	31,197	30,217	31,866	32,240	30,160	32,240	31,200	32,240	15,600	359,629

Esta configuración visual es un ejemplo de cómo no resumir un valor de instantánea. La suma de saldos de instantáneas diarias no produce un resultado significativo. Por lo tanto, elimine el campo **UnitsBalance** de la visualización.

- Ahora, agrega una medida que sume el valor de **UnitsBalance** para una sola fecha. La fecha será la última fecha de cada período de tiempo. Para ello utilice la función **LASTDATE**.

```
Stock on Hand = CALCULATE(
    SUM(Inventory[UnitsBalance]),
    LASTDATE('Date'[Date]))
```

Observa que la fórmula de la medida usa la función **SUM**. Se debe utilizar una función agregada (las medidas no permiten referencias directas a columnas), pero dado que solo existe una fila para cada producto para cada fecha, la función **SUM** solo operará en una sola fila.

- Añade la medida **Stock on Hand** a la visualización matriz de la página 2 del informe. El valor de **UnitsBalance** para cada producto ahora está basado en el último valor registrado para cada mes

Tema G: KPI (Key Performance Indicator)

Un **KPI** (Key Performance Indicator – Indicador Clave de Rendimiento) es un número basado en una medida específica del modelo de datos Power BI que permite evaluar el valor, estado y tendencia de una métrica. El KPI mide el valor de la métrica con respecto a un valor objetivo o valor de destino.

En la terminología empresarial, un KPI es una medida cuantificable que permite verificar el cumplimiento de los objetivos empresariales. Por ejemplo, un KPI para medir la utilidad bruta mensual frente a la utilidad bruta prevista, y otro KPI para evaluar los costos de producción comparando los gastos mensuales contra los ingresos.

Crear un KPI

Se ha establecido como objetivo del negocio que las ventas del mes deben ser superiores o iguales a las ventas del mes anterior. Diseñe un KPI que permita validar si el objetivo se está cumpliendo.

Ejercicio 1: Creación de las medidas para el cálculo del KPI

1. Abra el archivo **Adventure Works DW 2020 ModG.pbix**.
2. Agregue a la tabla **Sales** una columna calculada de nombre **Monto Item** que entregue el monto de cada ítem de la tabla luego de aplicarle el porcentaje de descuento (**Unit Price Discount Pct**). La expresión para evaluar el **Monto Item** es:

```
Monto Item =  
    Sales[Order Quantity]  
        * Sales[Unit Price]  
        * (  
            1 - Sales[Unit Price Discount Pct]  
        )
```

3. Añada a la tabla **Sales** una medida de nombre **Total Ventas** que calcule el monto total de las ventas. Utilice la siguiente fórmula:

```
Total Ventas = SUM('Sales'[Monto Item])
```

4. Añada a la misma tabla **Sales** una segunda medida de nombre **Total Ventas Mes Anterior** que calcule el monto total de las ventas del mes anterior usando la fórmula:

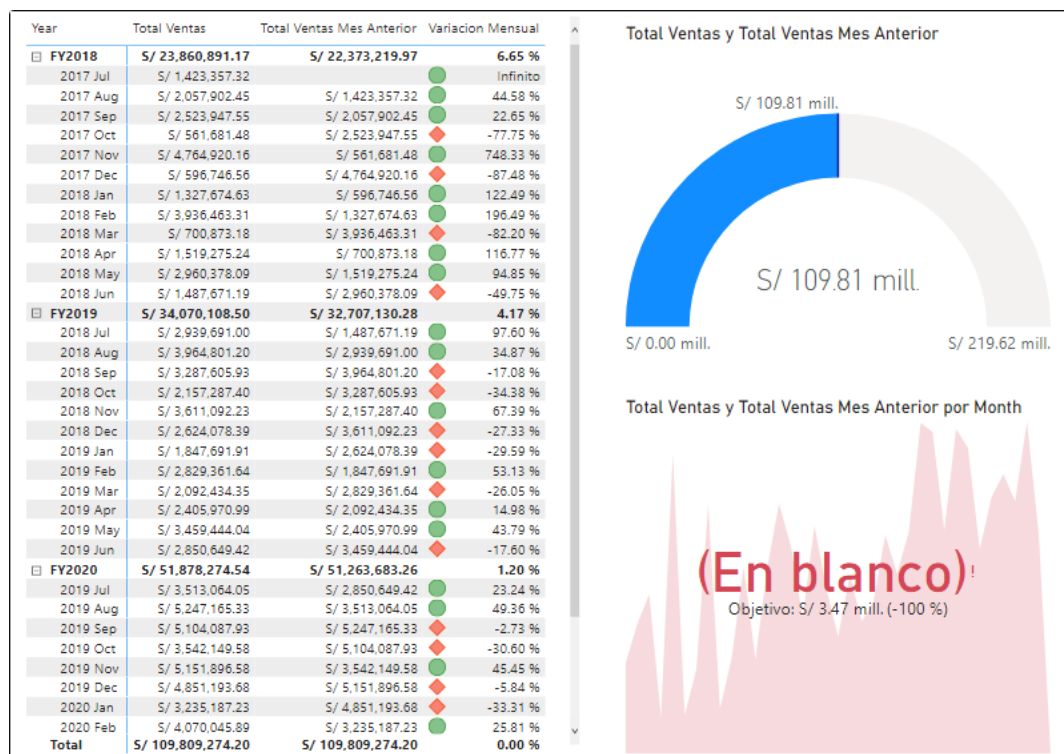
```
Total Ventas Mes Anterior = CALCULATE  
    ('Sales'[Total Ventas], DATEADD  
        ('Date'[Date], -1, MONTH  
    )  
)
```

5. Agregue a la tabla **Sales** otra medida de nombre **Variacion Mensual** que calcule la variación mensual de las ventas. Use la fórmula:

$$\text{Variacion Mensual} = \frac{([\text{Total Ventas}] - [\text{Total Ventas Mes Anterior}])}{[\text{Total Ventas Mes Anterior}]}$$

Ejercicio 2: Creación del informe Power BI

Despliegue el informe Power BI que se muestra a continuación.



Configurar "semáforo" en la columna Variacion mensual de la visualización matriz

1. En **Valores** de la visualización, clic en **Opciones de Variacion mensual, Formato condicional, Iconos**.
2. En la ventana **Iconos – Variacion mensual** defina:

Iconos - Variacion Mensual



Estilo de formato

Reglas

Aplicar a

Solo valores

¿En qué campo debemos basar esto?

Variacion Mensual

Diseño de los iconos

A la derecha de los datos

Alineación de los iconos

Superior

Estilo

Personalizado

Reglas

↑↓ Inversión del orden de lo...

+ Nueva regla

Si el valor	>	0	Número	y	<=	100	Porcentaje	entonces		↑ ↓ ×
Si el valor	=	0	Número					entonces		↑ ↓ ×
Si el valor	>=	0	Porcentaje	y	<	0	Número	entonces		↑ ↓ ×

[Más información sobre el formato condicional](#)

Aceptar

Cancelar